Assignment Report
on
Flipped Classroom
(MongoDB Foundations and Query Languages)

Subject Name: Big Data Analytics
Subject Code: BAD601

Submitted By:
T Abdul Siman(1AY23CD062)

Submitted To:
Ms. Surbhi

Table of Contents: -

Topic		Page No.
Day 1: Foundations of MongoDB (By Albin)		3-4
1.	Understood Concepts – Why MongoDB, MongoDB vs RDBMS, MongoDB Datatypes, Key Features (BSON/JSON Storage, Dynamic Querying, Sharding, Replication)	3
2.	Questions Asked	4
Day 2: MongoDB Query Language (By Sinchana)		5-6
1.	Understood Concepts – Insert, Save, Delete, Update, Find, Count, Skip, Sort Methods, Handling Null Values	5
2.	Queries discussed in class	6
3.	Questions Asked	6
Day 3: MongoDB Query Language (By Aswin)		7-8
1.	Understood Concepts – Aggregate, MapReduce Functions, JavaScript Programming in MongoDB, Cursors, Import/Export	7
2.	Queries Discussed in Class	8
3.	Questions Asked	8
Diagrams – Architecture and Query Pipeline in MongoDB		9

Day 1: Foundations of MongoDB (by Albin) – 11/03/2026

The presentation covered the basics of MongoDB and its importance in modern database systems:

MongoDB Overview:

MongoDB is a NoSQL, document-oriented database that stores data in flexible, JSON-like documents (BSON), unlike traditional table-based systems.

Why MongoDB:

It is preferred for modern applications due to:

High scalability (horizontal scaling using sharding)

Flexibility (no fixed schema)

High performance for large and unstructured data

- RDBMS vs MongoDB Terms: A comparison was made between traditional relational databases and MongoDB:
- Table → Collection
- Row → Document
- Column → Field
- Primary Key → _id

Data Types in MongoDB: MongoDB supports various data types such as strings, integers, arrays, objects, booleans, and dates, allowing complex and nested data structures.

Key Features:

- BSON Storage: Efficient storage using binary JSON format
- Dynamic Queries: Flexible querying without rigid schema constraints
- Replication: Copies data across servers to improve availability and fault tolerance
- Sharding: Distributes data across multiple servers for scalability.

Questions Asked:

1.How might MongoDB's document-based structure be more useful for modern applications?

MongoDB's document-based structure allows data to be stored in a flexible and nested format, which closely matches how data is used in real-world applications. This reduces the need for complex joins and makes data retrieval faster. It is especially useful for applications like web apps, mobile apps, and real-time systems where data structures frequently change. Developers can easily add or modify fields without redesigning the entire database schema.

2.Why do large systems often need both replication and sharding together?

Replication ensures that multiple copies of data are available across different servers, which improves fault tolerance and system reliability in case of failures. Sharding, on the other hand, distributes data across multiple machines to handle large volumes of data and high traffic efficiently. Large systems require both because replication alone does not improve performance under heavy load, and sharding alone does not guarantee data safety. Together, they ensure both scalability and high availability.

3.What advantages and possible challenges does flexible schema create?

Flexible schema allows developers to store different types of data in the same collection and easily adapt to changing requirements without major modifications. This speeds up development and is ideal for evolving applications. However, it can also lead to data inconsistency if not managed properly, as documents may have different structures. Developers need to enforce validation at the application level, which can increase complexity and require careful design.

Day 2: MongoDB Query Language (by Sinchana) – 12/03/26

The presentation focused on the MongoDB Query Language and how to perform various database operations:

MongoDB Query Language:

Introduction to how MongoDB uses methods to interact with collections and documents.

Insert Method:

Adding new documents using insertOne() and insertMany().

Save Method:

Used to insert or update documents depending on whether they already exist.

Update Method:

Modifying existing documents, including adding new fields or updating values using operators like $set.

Remove/Delete Method:

Deleting documents or specific fields using deleteOne() and deleteMany().

Find Method:

Retrieving documents based on conditions, filters, and search criteria.

Handling NULL Values:

Querying documents with null or missing fields.

Count, Limit, Sort, Skip: Used to manage query output:

- count() → total documents
- limit() → restrict results
- sort() → arrange order
- skip() → ignore initial results

Arrays in MongoDB:

- Working with array fields and querying elements within arrays.

Queries discussed with the class:

1. Insert a document into a collection named students. db.students.insertOne({ name: "Rahul", age: 20, course: "CSE" })
2. Find all documents where age is greater than 18. db.students.find({ age: { $gt: 18 } })
3. Update a student's course to "ECE" where name is "Rahul". db.students.updateOne({ name: "Rahul" }, { $set: { course: "ECE" } })
4. Delete a document where name is "Rahul". db.students.deleteOne({ name: "Rahul" })
5. Display only 5 documents sorted by age in descending order. db.students.find().sort({ age: -1 }).limit(5)

Questions Asked:

1. Why is the find() method considered one of the most important operations in MongoDB?

- The find() method is essential because it is used to retrieve data from the database.

2. What is the advantage of using update operators like $set instead of replacing the whole document?

- Using operators like $set allows updating only specific fields without affecting the rest of the document.

3. How do limit() and skip() help in real-world applications?

- These methods are commonly used for pagination in applications.

4. What challenges might arise when working with arrays in MongoDB?

- While arrays allow storing multiple values in a single field, querying and updating them can become complex, especially with nested arrays.

Day 3: MongoDB Query Language (By Aswin) – 13/03/26

The presentation focused on advanced MongoDB concepts and tools used for data processing and optimization:

Aggregate Function:

Explained how aggregation is used to process and analyze data, such as grouping, filtering, and calculating results using pipelines ($group, $match, etc.).

MapReduce Function:

Introduced as a method for processing large datasets by dividing tasks into smaller parts (map) and combining results (reduce).

JavaScript Programming in MongoDB:

Demonstrated how JavaScript can be used within MongoDB for custom queries and operations.

Cursors in MongoDB:

Explained how query results are returned using cursors, which allow iteration over documents.

Indexes:

Discussed how indexes improve query performance by allowing faster data retrieval.

MongoImport:

Used to import data from external files (like JSON or CSV) into MongoDB collections.

MongoExport:

Used to export data from MongoDB collections into external formats.

Automatic Generation of _id:

Explained how MongoDB automatically generates a unique identifier for each document.

Queries discussed with class:

1. Find the total number of students in each course using aggregation. db.students.aggregate([{ $group: { _id: "$course", total: { $sum: 1 } } }])
2. Create an index on the name field of the students collection. db.students.createIndex({ name: 1 })
3. Use a cursor to iterate through all documents in the students collection. var cursor = db.students.find(); while (cursor.hasNext()) { printjson(cursor.next()); }
4. Export the students collection to a JSON file. mongoexport --db=test --collection=students --out=students.json
5. Import data from a JSON file into the students collection. mongoimport --db=test --collection=students --file=students.json

Questions Asked:

1. Why is aggregation preferred over simple queries in MongoDB?

Aggregation allows performing complex data processing like grouping, filtering, and calculations in a single pipeline.

2. What is the difference between MapReduce and Aggregation?

Aggregation is faster and more optimized for most use cases, while MapReduce is more flexible and allows custom logic using JavaScript.

3. How do indexes improve database performance?

Indexes allow MongoDB to quickly locate data without scanning the entire collection.

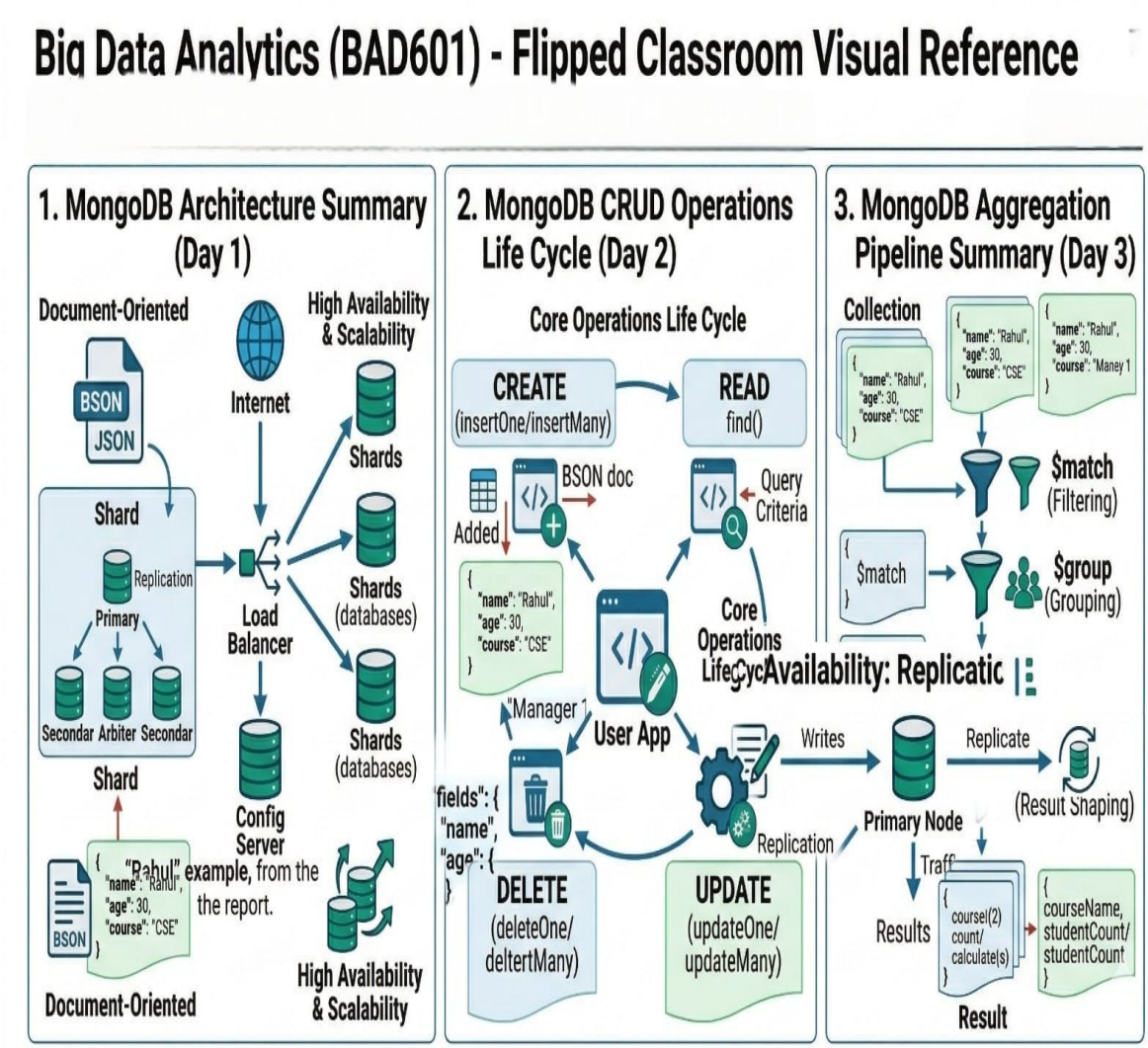

Fig. Representing MongoDB Architecture, Crud Operations and Pipeline